National Institute of Technology Karnataka

Department of Computer Science and Engineering

Multi-Signature Wallet Implementation using Ethereum Smart Contracts

CS740 - Blockchain Design Architecture

Submitted by:
Kunal Lagad (252IS019)
Sudeep Jain (252IS038)

Guided by:
Dr. Sourav Kanti Addya

Date: April 10, 2026

Contents

1 Abstract **2**

2 Introduction **3**

3 Problem Statement **4**

4 Objectives **5**

5 Technology Stack **6**

6 System Architecture **7**

7 Execution Flow of the System **8**

8 Implementation **9**

8.1 Smart Contract Implementation 9

8.1.1 Smart Contract Features 11

9 Results and Deployment **12**

10 Conclusion **15**

1 Abstract

Blockchain technology enables secure and decentralized management of digital assets. However, traditional cryptocurrency wallets rely on a single private key for authorization, which creates a single point of failure. If the private key is lost or compromised, the funds stored in the wallet may be permanently lost or stolen.

A multi-signature wallet is a type of cryptocurrency wallet that requires multiple approvals before a transaction can be executed. Instead of relying on a single owner, the wallet is controlled by multiple participants who must collectively approve transactions.

In this project, a multi-signature wallet was implemented using smart contracts on the Ethereum blockchain. The wallet allows multiple owners to manage funds and requires a predefined number of approvals before executing a transaction. The smart contract was developed using Solidity and deployed using Remix IDE connected to a local node running Geth. The implementation demonstrates secure collaborative fund management on a blockchain network.

2 Introduction

Blockchain technology has emerged as one of the most transformative innovations in modern computing. It provides a decentralized and distributed ledger system where transactions are recorded securely without the need for a central authority. Each transaction is grouped into blocks, and these blocks are cryptographically linked together to form a chain known as a a blockchain. Because each block contains the hash of the previous block, altering any transaction becomes extremely difficult. This feature ensures transparency, immutability, and trust within the network.

One of the most widely used applications of blockchain technology is cryptocurrency wallets. These wallets allow users to store, send, and receive digital assets such as Ether. Traditional cryptocurrency wallets are typically controlled by a single private key. The private key acts as proof of ownership and is required to authorize any transaction from the wallet. While this approach is simple, it creates a serious security risk. If the private key is lost or stolen, the funds stored in the wallet may become permanently inaccessible or may be transferred by unauthorized individuals.

To address these security challenges, multi-signature wallets were introduced. A multi-signature wallet requires multiple approvals before executing a transaction. Instead of relying on a single private key, the wallet is controlled by multiple owners who collectively manage the funds. A transaction can only be completed when a predefined number of owners approve it.

In this project, a multi-signature wallet is implemented using smart contracts on the Ethereum network. The smart contract enforces rules that require multiple confirmations before executing transactions. This implementation demonstrates how blockchain technology can enhance security and enable collaborative management of digital assets.

3 Problem Statement

Traditional cryptocurrency wallets rely on a single private key to control access to digital assets. While this design simplifies the transaction process, it introduces several security and operational challenges. If the private key is lost due to hardware failure, accidental deletion, or other unforeseen circumstances, the wallet owner permanently loses access to their funds. Unlike traditional banking systems, blockchain networks do not provide password recovery or account reset mechanisms.

Another major issue with single-key wallets is the risk of hacking or key theft. If an attacker gains access to the private key, they can immediately transfer all funds from the wallet without requiring any additional authorization. Because blockchain transactions are irreversible, recovering stolen funds becomes nearly impossible.

Single-key wallets also create difficulties in situations where multiple people need to manage shared funds. For example, in organizations, companies, or investment groups, financial decisions often require approval from multiple stakeholders. A traditional wallet controlled by a single key cannot enforce such collaborative decision-making.

The solution to these problems is a multi-signature wallet system. A multi-signature wallet requires multiple users to approve a transaction before it can be executed. This ensures that no single participant has complete control over the funds. By requiring multiple approvals, the system significantly reduces the risk of unauthorized transactions and improves overall security.

The objective of this project is to design and implement a multi-signature wallet using Ethereum smart contracts. The system should allow multiple owners to manage the wallet and require a predefined number of approvals before executing any transaction.

4 Objectives

The primary objective of this project is to implement a secure multi-signature wallet using blockchain technology. The system aims to demonstrate how smart contracts can be used to enforce transaction approval rules automatically and transparently.

One of the key objectives is to allow multiple users to jointly control a cryptocurrency wallet. Each user should have the ability to submit a transaction request and participate in approving transactions. However, no single user should be able to transfer funds independently.

Another objective is to ensure that transactions are executed only after receiving a predefined number of approvals from wallet owners. This requirement introduces an additional layer of security and reduces the likelihood of unauthorized transfers.

The project also aims to provide practical experience with Ethereum smart contract development. Students gain hands-on experience in writing smart contracts using Solidity, compiling and deploying them using Remix IDE, and interacting with them through blockchain transactions.

In addition to implementing the smart contract, the project also focuses on documenting the development process, demonstrating the working system, and analyzing transaction logs to verify successful execution.

5 Technology Stack

The implementation of the multi-signature wallet relies on several technologies that support blockchain development and smart contract execution.

The primary platform used in this project is Ethereum. Ethereum is a decentralized blockchain platform that supports programmable smart contracts. Unlike traditional blockchains that only record transactions, Ethereum allows developers to create decentralized applications that execute automatically when certain conditions are met.

The smart contract for the wallet is written using Solidity. Solidity is a high-level programming language designed specifically for writing smart contracts on Ethereum. It supports features such as contract inheritance, complex data structures, and event logging, which are essential for building decentralized applications.

For development and testing, Remix IDE was used. Remix provides a browser-based environment for writing, compiling, and deploying smart contracts. It also allows developers to test contracts using built-in debugging and transaction monitoring tools.

A local blockchain network was created using Geth, which is an implementation of the Ethereum protocol written in Go programming language. Geth allows developers to run a private Ethereum network for testing smart contracts without using the public Ethereum network.

The project source code and documentation were managed using GitHub, which provides version control and collaborative development features.

6 System Architecture

The multi-signature wallet system follows a decentralized architecture where the smart contract acts as the central control component. The architecture consists of wallet owners, a smart contract, the Ethereum blockchain network, and the transaction execution mechanism.

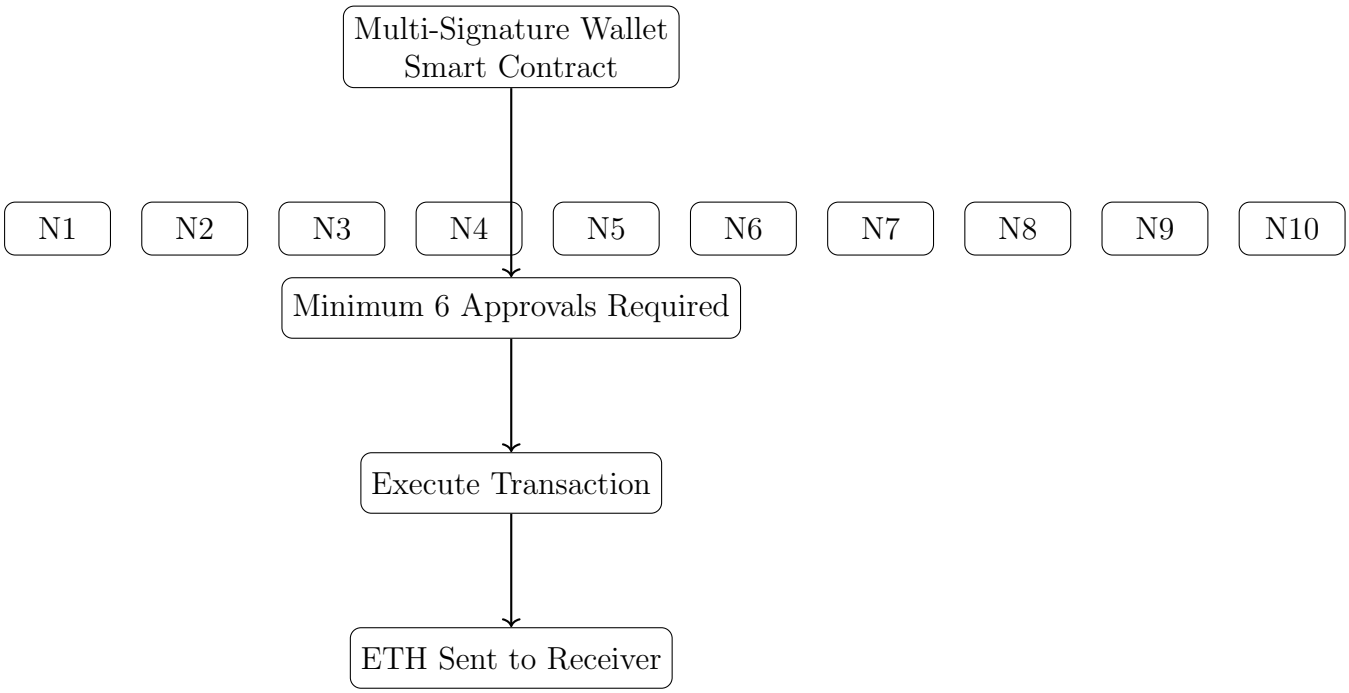

Figure 1: System Architecture of Multi-Signature Wallet

Multiple wallet owners are defined during the deployment of the smart contract. These owners represent the participants who collectively manage the wallet funds. Each owner has the ability to submit transactions and approve transaction requests.

When a wallet owner wants to transfer funds, a transaction request is submitted to the smart contract. The smart contract records this request and marks it as pending. The transaction details include the destination address, transfer amount, and execution status.

Other wallet owners can review the pending transaction and submit their approvals. The smart contract keeps track of approvals for each transaction and ensures that the same owner cannot approve the transaction more than once.

Once the required number of approvals is reached, the smart contract allows the transaction to be executed. During execution, the contract transfers the specified amount of Ether to the destination address. The transaction is then recorded permanently on the blockchain.

This architecture ensures transparency and security because every action is recorded on the blockchain and verified by the smart contract logic.

7 Execution Flow of the System

The execution flow of the multi-signature wallet describes the step-by-step process from contract deployment to transaction execution.

The process begins with writing the smart contract code in Solidity. The contract defines wallet owners, approval requirements, and functions for submitting, approving, and executing transactions.

Once the code is written, the contract is compiled using the Solidity compiler in Remix IDE. Compilation converts the contract code into bytecode that can be deployed on the Ethereum blockchain.

After compilation, the contract is deployed on a local blockchain network using Remix connected to a Geth node. Deployment creates a smart contract instance and assigns it a unique blockchain address.

Users can then deposit Ether into the wallet by sending funds to the contract address. The contract stores these funds and updates its internal balance.

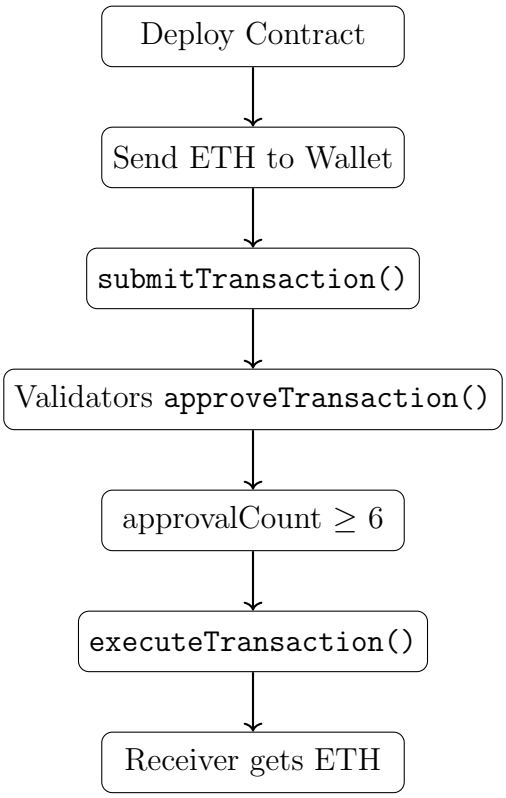

Figure 2: Execution Flow of Multi-Signature Wallet System

When a wallet owner wants to transfer funds, they create a transaction request using the `submitTransaction()` function. The transaction is stored in the contract as a pending transaction.

Other wallet owners approve the transaction using the `approveTransaction()` function. Each approval increases the confirmation count.

Once the required number of approvals is reached, the `executeTransaction()` function can be called to transfer the funds to the specified recipient.

8 Implementation

The implementation of the multi-signature wallet involves designing and deploying a smart contract that enforces the multi-approval mechanism.

The contract maintains a list of wallet owners and the minimum number of approvals required for executing transactions. It also stores a list of transaction requests and tracks the approval status for each transaction.

When a new transaction is submitted, the contract records the transaction details and sets its execution status to false. Wallet owners can then review the transaction and approve it using their accounts.

Each approval is recorded in the smart contract. The contract checks whether the transaction has received enough approvals before allowing execution.

Once the required number of approvals is reached, the `executeTransaction()` function transfers the specified amount of Ether from the contract to the recipient address.

Screenshots of the smart contract code, compilation results, contract deployment, transaction submission, approval process, and execution results are included in the report to demonstrate successful implementation.

8.1 Smart Contract Implementation

The following Solidity smart contract implements the multi-signature wallet used in this project. The contract manages wallet validators, transaction submissions, approvals, and execution after reaching the required number of confirmations.

Listing 1: Multi-Signature Wallet Smart Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract MultiSigWallet {

    address[] public validators;
    uint public requiredApprovals;

    struct Transaction {
        address to;
        uint amount;
        bool executed;
        uint approvalCount;
    }

    Transaction[] public transactions;

    mapping(address => bool) public isValidator;
    mapping(uint => mapping(address => bool)) public approvals;

    constructor(address[] memory _validators, uint
       _requiredApprovals) {

        require(_validators.length == 10, "Need exactly 10
           validators");
```

```
        require(_requiredApprovals <= _validators.length, "
           Invalid approval count");

        for(uint i=0;i<_validators.length;i++){
            address validator = _validators[i];
            validators.push(validator);
            isValidator[validator] = true;
        }

        requiredApprovals = _requiredApprovals;
    }

    modifier onlyValidator(){
        require(isValidator[msg.sender], "Not validator");
        _;
    }

    function submitTransaction(address _to, uint _amount) public
       onlyValidator {

        transactions.push(Transaction({
            to: _to,
            amount: _amount,
            executed: false,
            approvalCount: 0
        }));
    }

    function approveTransaction(uint txId) public onlyValidator {

        Transaction storage txn = transactions[txId];

        require(!txn.executed, "Already executed");
        require(!approvals[txId][msg.sender], "Already approved")
           ;

        approvals[txId][msg.sender] = true;
        txn.approvalCount++;

        if(txn.approvalCount >= requiredApprovals){
            executeTransaction(txId);
        }
    }

    function executeTransaction(uint txId) public {

        Transaction storage txn = transactions[txId];

        require(!txn.executed, "Already executed");
        require(txn.approvalCount >= requiredApprovals, "Not
           enough approvals");
```

```
        txn.executed = true;

        payable(txn.to).transfer(txn.amount);
    }

    receive() external payable {}

}
```

8.1.1 Smart Contract Features

The smart contract implements a secure multi-signature wallet system using Ethereum smart contracts. The main components of the contract are described below:

Validators List

The contract maintains a list of wallet validators who are authorized to submit and approve transactions. Only these validators are allowed to interact with critical wallet functions.

Transaction Structure

Each transaction contains the destination address, the transfer amount, execution status, and the number of approvals received. These transactions are stored in a transaction list within the smart contract.

Approval Mechanism

Validators approve transactions using the `approveTransaction()` function. The contract ensures that a validator cannot approve the same transaction multiple times.

Execution Mechanism

Once the number of approvals reaches the required threshold, the `executeTransaction()` function is triggered. The smart contract then transfers the specified amount of Ether to the recipient address.

Security Enforcement

The contract enforces access control using the `onlyValidator` modifier and ensures that transactions cannot be executed without sufficient approvals. This mechanism significantly improves the security of digital asset management.

9 Results and Deployment

The multi-signature wallet system was successfully implemented and tested on a local Ethereum network. The smart contract allowed multiple owners to manage wallet funds collectively and required multiple approvals before executing transactions.

During testing, transactions were created and approved by different wallet owners. The system correctly tracked the number of approvals and prevented duplicate approvals from the same owner.

Transactions were executed only after reaching the required number of confirmations. The transaction logs displayed in the Remix console and Geth terminal confirmed that gas was consumed and the contract state was updated.

These results demonstrate that the multi-signature wallet successfully enforces collaborative decision-making and enhances the security of digital asset management.

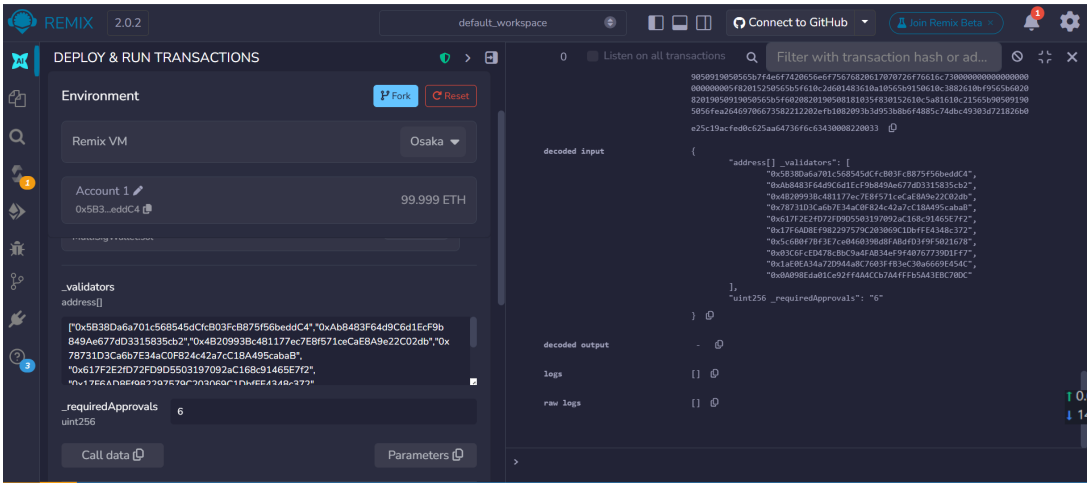

Figure 3: Smart Contract Deployment with Validator Addresses

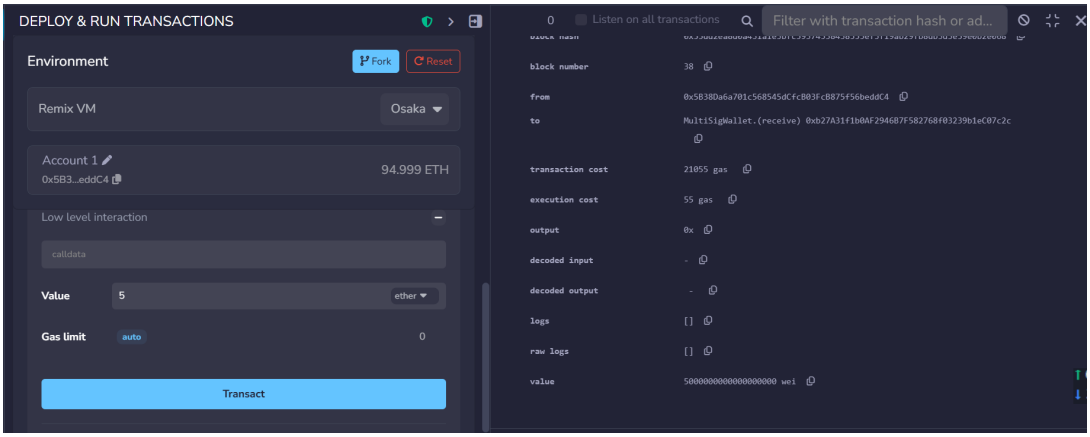

Figure 4: Funding the Multi-Signature Wallet Smart Contract with 5 ETH

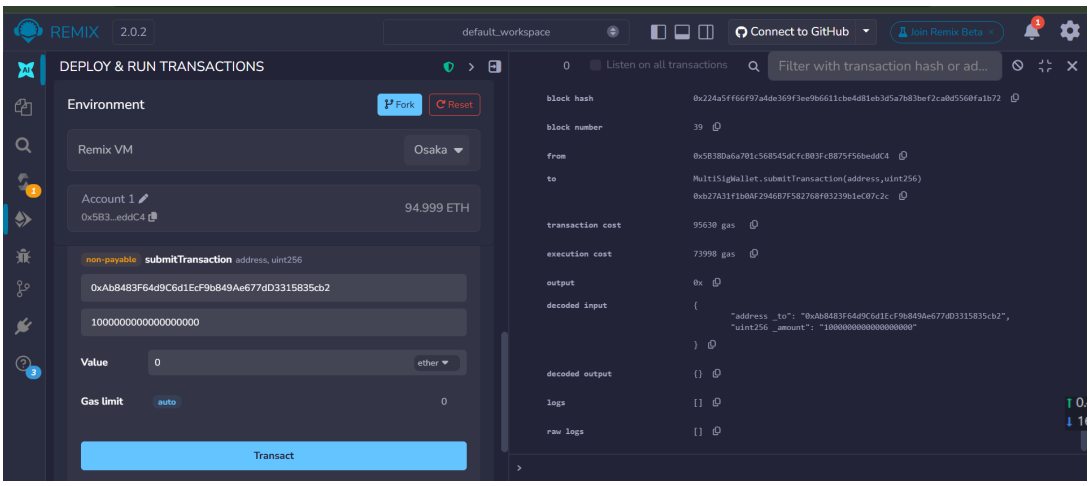

Figure 5: Submitting a Transaction Request

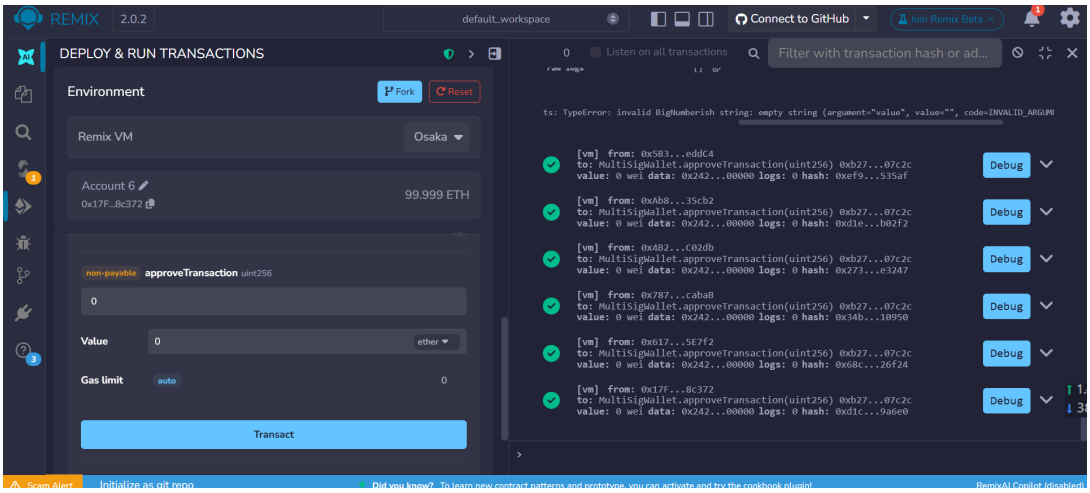

Figure 6: Validator Approval of Transaction Using approveTransaction() Function

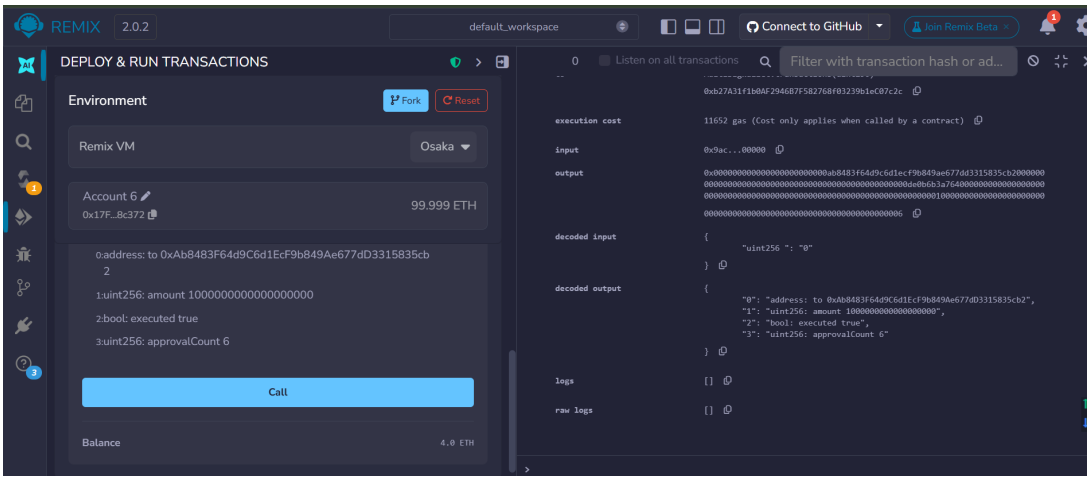

Figure 7: Verification of Transaction Status Using transactions() Function

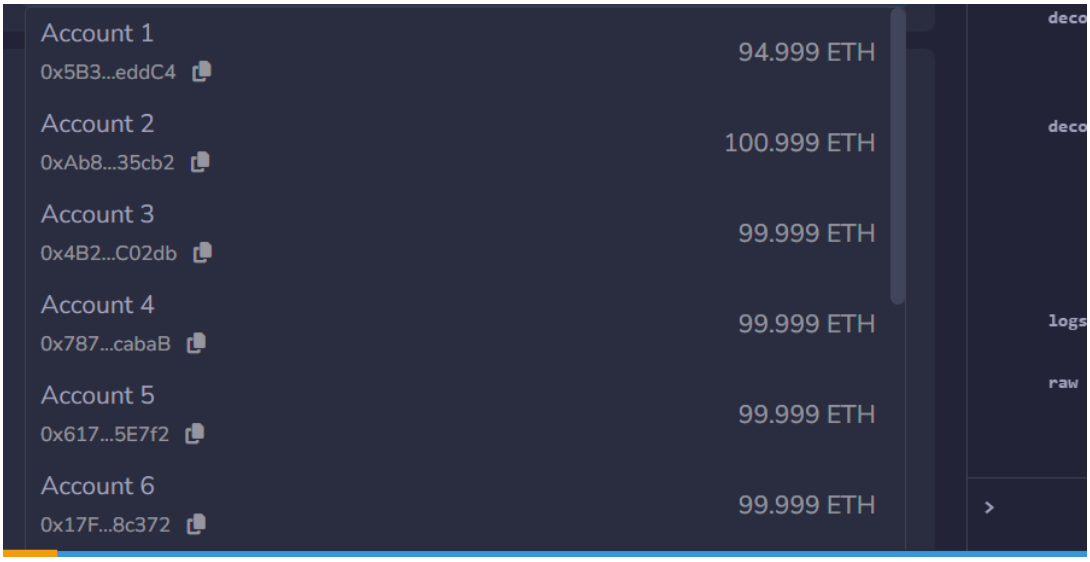

Figure 8: Verification of Transaction Status

10 Conclusion

This project demonstrated the design and implementation of a multi-signature wallet using Ethereum smart contracts. The system successfully enforced a multi-approval mechanism for transaction execution, ensuring that funds could only be transferred after receiving sufficient approvals from wallet owners.

The use of smart contracts allowed automatic enforcement of security rules without relying on a central authority. The implementation also provided practical experience with Ethereum development tools and blockchain transaction management.

Multi-signature wallets are widely used in real-world blockchain applications because they provide enhanced security and enable collaborative control of digital assets.